QINBIR

■■■■ | Automatic University Course Scheduling System

PROJECT DOCUMENTATION
Concise technical overview

Version	1.0	**Date**	November 2026
Frontend	React + Vite	**Backend**	Python + FastAPI
Database	SQLite / PostgreSQL	**Optimization**	PuLP + CBC

A centralized platform for managing university academic resources and generating conflict-aware course timetables.

GitHub • Vercel • Render

1. Project Overview

QINBIR is a web-based university course scheduling system. It centralizes academic scheduling data and uses mathematical optimization to assign courses to lecturers, student sections, rooms, and time slots. It is designed to reduce manual work and prevent timetable conflicts.

Main objectives

• Automate timetable generation and reduce scheduling errors.

• Manage departments, programs, courses, sections, lecturers, rooms, and time slots.

• Record lecturer availability and course session requirements.

• Validate schedules before publication.

• Give lecturers and students access to their own published timetable information.

2. Users and Permissions

Role	Main capabilities
Administrator	Manage academic data, scheduling constraints, generate, validate, and publish schedules.
Lecturer	View their own published teaching timetable, assigned courses, sections, rooms, and times.
Student	View their assigned section and its published classes, lecturers, rooms, and times.

3. Technology and Architecture

Layer	Technology	Responsibility
Frontend	React, Vite, JavaScript, CSS	Dashboards, forms, tables, user interaction.
Backend	Python, FastAPI, SQLAlchemy, Pydantic	API, business rules, authentication, data access.
Database	SQLite / PostgreSQL	Stores users and scheduling records.
Optimizer	PuLP with CBC solver	Builds and solves scheduling optimization models.

4. Core Modules

Academic data management

Departments, programs, courses, student sections, lecturers, and classrooms.

Scheduling resources

Time slots and lecturer availability.

Course requirements

Connects a course, lecturer, and student section; records sessions per week and session durations.

Constraints

Hard constraints must be satisfied; soft constraints express preferences the optimizer should try to satisfy.

Schedule management

Generate, validate, review, and publish schedules.

Personal timetables

Role-protected lecturer and student views of published timetable entries.

5. Scheduling and Validation

The scheduling engine formulates course assignments as an optimization problem. It considers course requirements, lecturer availability, student sections, rooms, time slots, and configured constraints. The CBC solver searches for a feasible or optimal assignment according to the model objective.

Typical workflow

Load academic data	→	Build optimization model	→	Solve assignments
Validate conflicts	→	Save draft schedule	→	Administrator publishes

Validation checks include lecturer clashes, room clashes, student-section clashes, and course requirement/session coverage.

6. Key API Endpoints

Method	Endpoint	Purpose
POST	/auth/login	Authenticate user.
GET	/auth/me	Get authenticated account.
GET	/health	Backend health check.
GET	/timetable/lecturer/me	Get current lecturer's published timetable.
GET	/timetable/student/me	Get current student's section timetable.
POST	/schedules/{schedule_id}/publish	Publish a draft schedule.

7. Setup and Running Locally

Prerequisites: Python 3.12, Node.js/npm, Git, and a local database configuration.

Backend (PowerShell)

```
cd C:\Users\Hp\documents\qinbir\backend
.\.venv312\Scripts\Activate.ps1
pip install -r requirements.txt
python -m uvicorn app.main:app --reload
```

Frontend (second PowerShell window)

```
cd C:\Users\Hp\documents\qinbir\frontend
npm install
npm run dev
```

Local addresses

Frontend: http://localhost:5173 (or the Vite URL shown in the terminal)
API: http://127.0.0.1:8000
API docs: http://127.0.0.1:8000/docs

8. Deployment

Service	Platform	Address
Source code	GitHub	github.com/LombameLemma/QINBIR
Frontend	Vercel	qinbir-tt4t.vercel.app
Backend	Render	qinbir.onrender.com

A push to the connected GitHub branch may trigger automatic deployments. Confirm deployment settings in Vercel and Render. Code deployment does not itself replace the production database.

9. Verified Project Results

A previously published schedule was checked with 90 entries. Validation reported zero lecturer, room, or student-section clashes. The tested schedule represented 50 course requirements with no missing, unknown, or invalid session patterns. Lecturer and student personal timetable endpoints both returned successful responses during local testing.

10. Security and Next Steps

The application uses JWT bearer authentication and role-based access checks. Keep secrets and local database backups out of Git. Back up production data before schema changes, and use controlled database migrations for future updates.

Potential improvements

PDF/Excel timetable export, stronger automated tests, department-by-department scheduling, improved room utilization, schedule versioning, and notification support.

Project links
GitHub: https://github.com/LombameLemma/QINBIR
Frontend: https://qinbir-tt4t.vercel.app
Backend/API: https://qinbir.onrender.com
API documentation: https://qinbir.onrender.com/docs